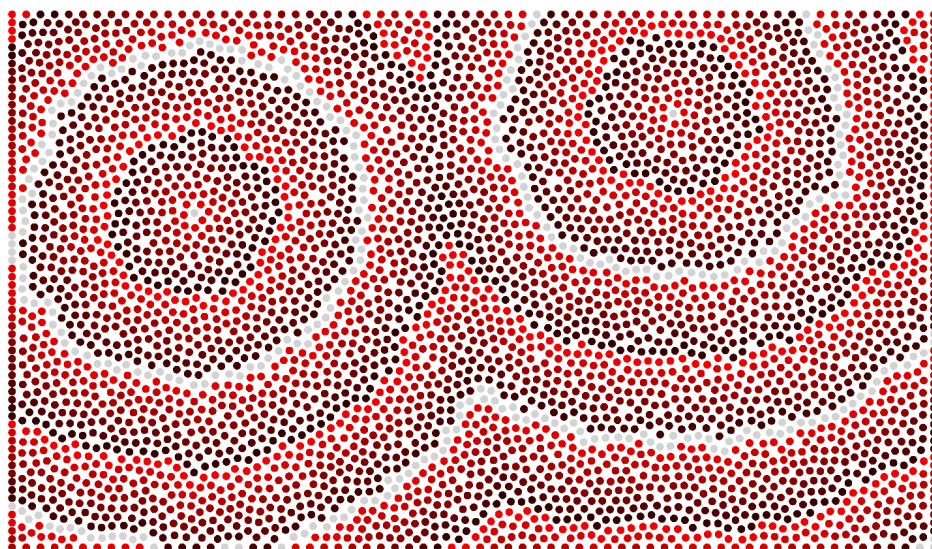


Université Paris-Saclay UFR des Sciences d'Orsay Computer Science Department
Master 2 Quantum and Distributed Computer Science (QDCS)
2025-26 internship Report

BLOB COMPUTING IN HOMOGENEOUS ISOTROPIC DISTRIBUTED SYSTEMS



an internship authored by

Lucas Labouret

performed at the

Laboratoire Interdisciplinaire des Sciences du Numériques

and supervised by

Frédéric Gruau

25 August, 2026

Abstract

The current trend in processor design is to miniaturize the transistors to increase the amount that can fit on a single chip. Following Moore's law, this approach has been very successful so far. However, as we approach the physical limit of how small a transistor can be, we need to find new ways to increase the computational power of our systems. A recent example is a new type of chip produced by IBM, with nodes of 0.7nm, or roughly 15 rows of silicon atoms [5]. In this internship we explore a massively parallel computational paradigm called Blob computing, which is based on the interactions of simple, locally communicating processing elements arranged in space. The spatial nature of this paradigm allows us to design algorithms that are naturally suited to the physical constraint that the propagation time of information scales linearly with distance. Through the inherent spatial nature of this paradigm, we aim to achieve arbitrarily scalable computational power by simply increasing the number of processing elements in the system. This work focuses on homogeneous and isotropic connection topologies, which generalize previous studies based on regular lattices, but also present significant new challenges in the design of algorithms for this paradigm.

Keywords: Distributed systems. Natural computing. Cellular automata.

Contents

1	Introduction	1
1.1	Presentation of the host organization	1
1.2	Internship context and objectives	2
1.3	Zoom on the Blob computing paradigm	2
1.3.1	Blob Computing on the micro level	2
1.3.2	Blob Computing on the macro level	4
1.3.3	About the media	5
2	Performed work	7
2.1	Initial Direction	7
2.2	Interpreter and Blob Language	7
2.2.1	Interpreter	8
2.2.2	Blob language	8
2.2.3	Bit packing and Parallelization	11
2.3	Graphical User Interface	11
2.4	Preliminary results	12
2.5	Toward a solution to the homogenization problem	14
2.5.1	Dynamic distance	15
2.5.2	Gabriel centers	15
2.5.3	Computation in the hexagonal lattice case, and challenges in the homoge- neous isotropic case	16
2.5.4	Current attempts at solving the problem	17
3	Evaluation and discussion	20
3.1	Evaluation of the performed work	20
3.2	Impact of the internship	20
3.3	Reflections on the internship	21
3.3.1	Difficulties encountered and reactions	21
3.3.2	Skills acquired and lessons learned	21
3.3.3	Career plans after the internship	21
4	Conclusion	22
4.1	Acknowledgements	22

References	22
Appendices	23
A List of built-in Blob instructions	24

Chapter 1

Introduction

Blob computing is a programming and computational framework inspired by cellular automata (CA), designed to make spatially distributed computation arbitrarily scalable and more accessible.

As we reach the limit of miniaturization, we will need to make chips larger to increase their computational power. We aim to provide a computational model where the impact of physical distance between processing elements (PEs) is inherently accounted for rather than being an afterthought, as is the case in most current programming paradigms. This will allow us to design algorithms that are more efficient and better suited to physical constraints.

While standard CA can exhibit complex behaviors, it can be extremely difficult to program those behaviors. Blob computing aims to address this issue by providing a higher-level programming model that allows users to manipulate "blobs" of PEs rather than each individual PE. This allows for powerful abstractions that make programming a lot more intuitive.

In this report, we present work carried out during the internship in two directions. The first concerns the design of a simple Blob language, its interpreter, and a graphical user interface for visualizing and interacting with Blob programs. The second concerns an initial study of the homogenization problem in homogeneous isotropic distributed systems, which consists in evenly spreading particles in a medium and is closely related to load balancing in distributed systems. We describe the first results obtained on this problem, the difficulties encountered, and possible directions for future work. The code developed during this internship is available at <https://github.com/Lucas-Labouret/BlobComputing>.

Note : While the defense of this internship is set to happen on September 2nd, my internship will only be completed by October 23rd, leaving me with a little more than a quarter of the total time of the internship to complete my work. Thus, the information presented in this report is not final, and may be subject to change as I continue to work on the project.

1.1 Presentation of the host organization

The Laboratoire Interdisciplinaire des Sciences du Numériques (LISN) is a research laboratory in computer science situated in Orsay, France. It is part of the Université Paris-Saclay and the CNRS. It employs a total of 180 researchers, 54 technicians and engineers, and 145 PhD students. It also welcomes 200 internships each year.

The research efforts are distributed in 17 teams over 5 poles. The research topics covered by the laboratory are very diverse, including algorithms, distributed systems, machine learning, data science, and human-computer interaction, and constitute a rich environment for interdisciplinary work. In particular, the pole "Algorithms, Learning and Computation", which encompasses research teams working on distributed systems and cellular automata is the perfect environment for this internship.

I was supervised by Associate Professor Frédéric Gruau, who is not currently affiliated with any of the research teams, but has a long history of research in cellular automata and natural computing.

1.2 Internship context and objectives

This internship is the continuation of my M1 internship in 2025, during which I worked on a parallel algorithm for simulating Blob operations on a single machine. The present internship builds on that work and is organized around three objectives:

- Define a language for writing Blob programs and implement an interpreter that reuses the parallel simulation techniques developed during my M1 internship.
- Develop a graphical user interface to visualize the execution of Blob programs and interact with the simulation.
- Design an algorithm for the homogenization problem in homogeneous isotropic distributed systems and evaluate its performance.

The homogenization problem is interesting in three ways. Firstly, it is closely related to load balancing, a fundamental problem in distributed systems. Secondly, it is a spatial problem (i.e. it is about the spatial arrangement of the elements in the system), which makes it a natural fit for the Blob computing paradigm. Finally, it is a self-organization problem, as we essentially want the particles to organize using only the local information available to them.

1.3 Zoom on the Blob computing paradigm

While we give a fairly detailed description of the basics of Blob computing in this section, we refer the reader to [2] and [1] for a more complete and formal presentation.

1.3.1 Blob Computing on the micro level

In blob computing, computation is performed by a set of simple, locally interacting agents, called "processing elements" (PEs). These PEs all execute the same sequence of operations synchronously. They are arranged in space following the topology of a Delaunay triangulated embedded planar graph. The set of all PEs is called a "computing medium", or simply "medium". The PEs are of different types, depending on their position in the graph. Each PE can only directly communicate with its neighbors in the graph, and the type of communication depends on the types of the PEs involved.

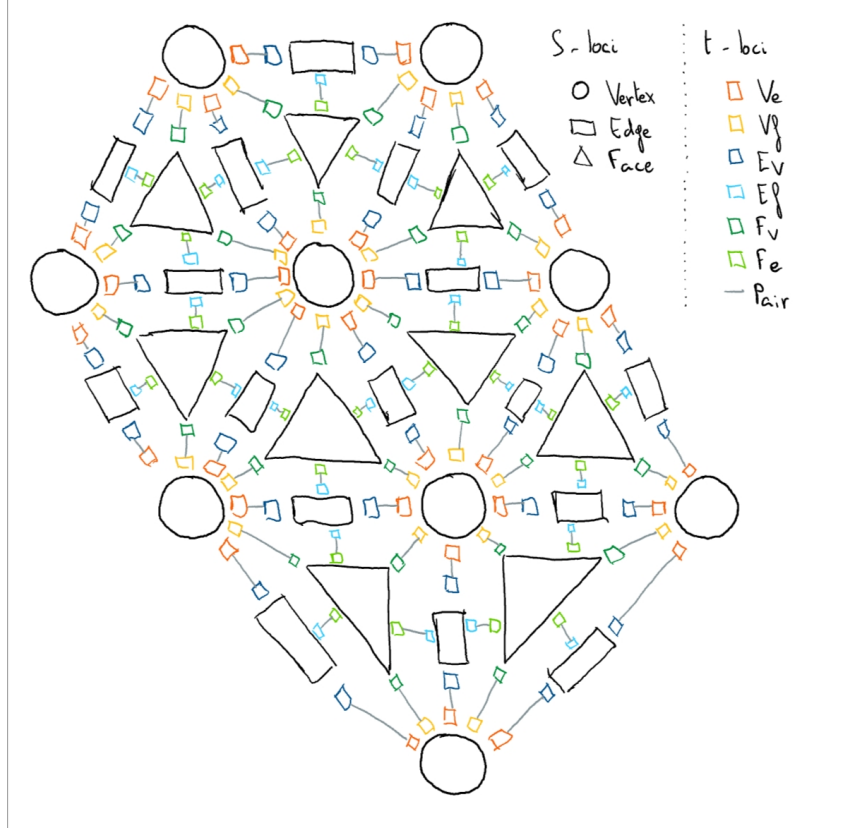


Figure 1.1: A simple example of a medium.

There are 9 distinct types of PEs :

- 3 simplicial types that correspond to the vertices, edges, and faces (excluding the exterior face) of the graph
- 6 transfer types. These are used to transfer information between the simplicial PEs. They are defined by the type of simplicial PE they are transferring from and to. For example, a transfer PE of type "Ve" ("vertex to edge") is used to transfer information from a vertex PE to an edge PE.

PEs are restricted to a very limited set of operations :

- **Broadcasts** : a simplicial PE can broadcast a message to its neighboring transfer PEs of a given type. For example, a *Vertex* can broadcast a message to its neighboring *Ves*, and a *Face* can broadcast a message to its neighboring *Fes*. Importantly, *this is the only way of direct communication from simplicial to transfer PEs*, it is not possible for a simplicial PE to single out a transfer PE to communicate exclusively with.
- **Logical reductions** : a simplicial PE can perform a reduction operation on the messages received from its neighboring transfer PEs of a given type. For example, a *Vertex* can receive a message that is the result of a successive OR (resp. AND, XOR) of the messages sent by its neighboring *Ves*. Similarly to broadcasts, this is the only way of direct communication from transfer to simplicial PEs.

- **Transfers** : a transfer PE can send a message to its dual transfer PE. For example, a *Ve* can send a message to its dual *Ev*.
- **Rotations** : a simplicial PE can "rotate" a message clockwise or counterclockwise between its neighboring transfer PEs of a given type. For example, a *Vertex* can rotate a message from its *Ves* to its *Vfs* (and vice versa), and from its *Vfs* to its *Ves*.
- **Logical operations** : any PE can perform logical operations (AND, OR, XOR) on any bits of its state.

1.3.2 Blob Computing on the macro level

What makes Blob computing interesting is the emergence of complex behaviors from the interactions of simple PEs. By carefully designing the local rules that govern the behavior of the PEs, it is possible to achieve global behaviors that are useful for solving computational problems.

Unsurprisingly, Blob computing focuses on the manipulations of "blobs", which are connected sets of PEs of the same type in the medium. This makes it easier to reason about the system at a higher level. As an example, it is much easier to visualize a ball drifting in a medium than to visualize the individual vertices that make up the ball filling and emptying their neighbors while trying to maintain the shape of the ball.

The connexity of blobs is an interesting property for two reasons. Firstly, it can be preserved locally while still having global implications. Secondly, it provides a way to track the evolution of the blob over time.

For example, it is fairly easy to compute the discrete Voronoï diagram of a set of static particles in the medium, which is a global property, by having the blob of each particle grow on its neighboring empty vertices (which preserves connexity), unless it would merge with another blob (which would break the non-connexity of distinct blobs).

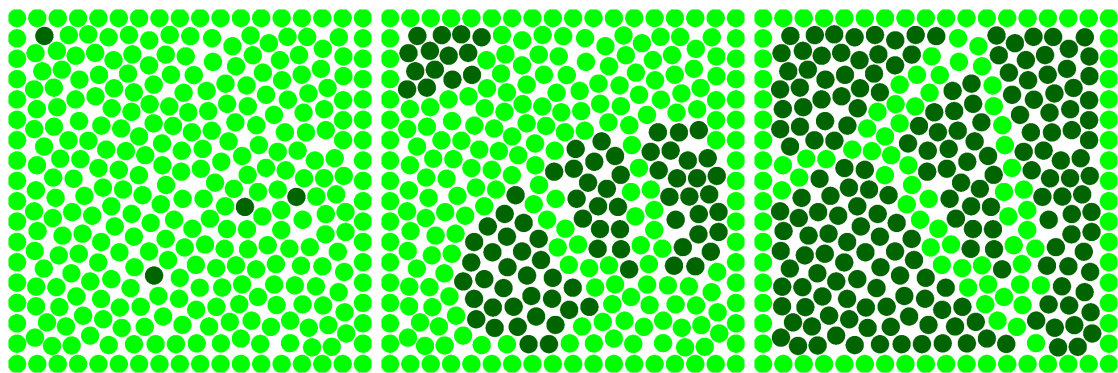


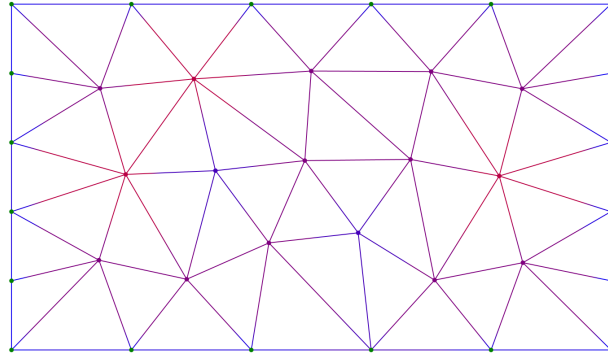
Figure 1.2: 3 snapshots of the Voronoï computation of 4 particles in a medium. Blobs are represented in dark green, while empty vertices are light.

While the Voronoï computation is fully spatial, it is also possible to perform numerical computations. For example, at least in the case of a hexagonal lattice medium, it is possible to compute the product of two matrices by having a blob repeatedly split vertically and horizontally to create a grid of blobs of suitable size, then apply a systolic algorithm to compute the product of the two matrices.

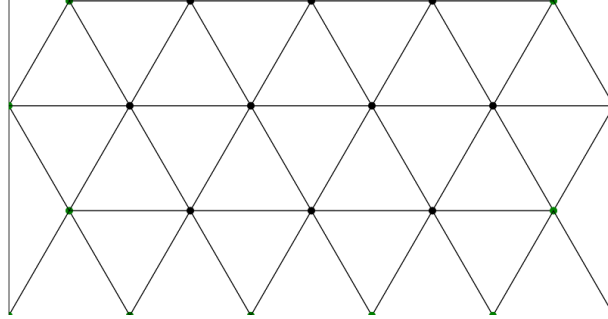
1.3.3 About the media

In this internship I focused on a very specific kind of medium : homogeneous and isotropic with a regular rectangular border. A medium is homogeneous if the vertices of the underlying embedded graph are distributed in a way that minimizes (or is close to minimizing) the maximal distance between any two vertices. It is isotropic if the direction of the edges follows no preferential direction (i.e. the distribution of the angles of the edges is close to uniform).

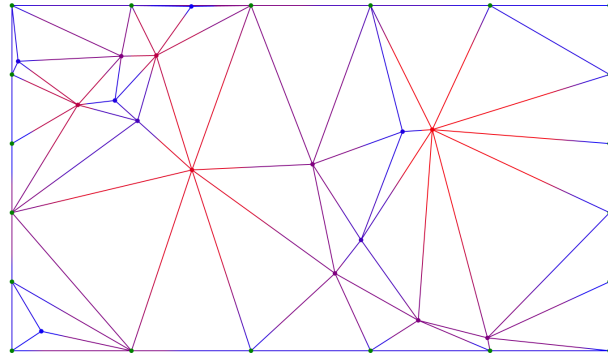
We accepted a small caveat regarding the homogeneity and isotropy of the medium : we allowed the border to be a perfect rectangle with regularly spaced vertices. Such a border is very helpful in controlling strange behaviors that can arise near it, because it allows us to easily "trick" the border PEs into behaving as if there was a perfect mirror image of the medium on the other side of the border. This mirror image is useful for properly defining the behavior of the medium near the border.



(a) Homogeneous and isotropic.



(b) Homogeneous but not isotropic.



(c) Isotropic but not homogeneous.

We chose to focus on this specific kind of medium for two reasons. Firstly, this is a natural generalization of the hexagonal lattice medium, while still being similar enough that we can

hopefully easily adapt existing ideas to this new setting. Secondly, letting go of the homogeneity would not make much sense in the context of maximizing chip area usage, as it would both be a waste of space to have large areas of the chip with no PEs, and increase the communication time by increasing the distance between the PEs.

I generated these media last year as part of my M1 internship, and was able to generate homogeneous and isotropic media with up to 4,355 vertices. Even though this may seem low, such media contain about 12,000 edges and 25,000 *Ves*. Ultimately, given that I was running all these experiments on my relatively weak personal computer, I decided that 4,355 vertices was enough for the scope of this project, and that building a larger medium would not be worth the time.

The code used to generate these media and to setup the parallelization alluded to in 2.2.3 is available at <https://github.com/Lucas-Labouret/MilieuCalcul>.

Chapter 2

Performed work

2.1 Initial Direction

The first few weeks of the internship were spent familiarizing myself with Frédéric's current Blob computing framework, which is based on hexagonal media. Our intent was to extend this framework to support homogeneous and isotropic media so as to reuse as much of the existing code as possible. We ultimately decided to abandon this approach, as it would have required a significant amount of work to adapt the existing code to the new medium, which would have stopped us from working on the homogenization problem. Instead, we decided that it would be more efficient to quickly develop a new framework from scratch.

During this time, I also wrote simple mock ups of the algorithms that would have to be implemented in the old framework to adapt it to the new media. This allowed me to quickly test the algorithm and fix several bugs in a simpler setting. While I did not end up modifying the old framework, this was not a waste of time as this mock implementation ended up being the basis of the new Blob interpreter (see [2.2](#)).

2.2 Interpreter and Blob Language

The blob language is a simple, high-level language that allows users to write Blob programs in a convenient way. It is based on the notion of field programming : primitive data types are maps from the PEs of a given type to values. Specifically, we only use boolean fields, which are maps from the PEs of a given type to boolean values. A Blob program is then a sequence of instructions that manipulate these fields. It is similar in spirit to an automaton, where the state of the medium at a given time is represented by a collection of fields, and the program defines a transition function that takes the current state as an input and produces the next state.

This field programming paradigm is very natural in the context of blob computing : since PEs act synchronously, any operation that can be performed by a PE can be easily expressed as an operation on the field of values of that PE type. For example, a rotation $Ve \leftarrow Vf$ can be expressed as an operation that takes the field of values of the Ves and produces a new field of values for the Vfs , where the value of each Vf is equal to the value of the Ve that is rotated to it. Similarly, a broadcast from *Vertex* to *Ves* can be expressed as an operation that takes the field of values of the *Vertices* and produces a new field of values for the *Ves*, where the value of

each Ve is equal to the value of the *Vertex* that is broadcasting to it.

Because of the limited time available during this internship, I decided to develop a simple language that sits on top of Java. This allows us to avoid the overhead of developing a full-fledged programming language. In particular, this allows us to forgo the development of a lexer and a parser, and to easily make the Blob language strongly typed and object oriented, since we can directly reuse Java's built-in features for this purpose.

2.2.1 Interpreter

The interpreter is built around an *Instruction* interface, which declares a single method, `exec()`. Calling this method performs a single step of the instruction, then returns a boolean indicating whether the instruction has finished executing or not. The *Instruction* interface has two direct implementations : the *BasicInstruction* interface, which is implemented by single-step instructions, and the *Procedure* abstract class, which is used to create composite instructions. Basic instructions are the building blocks of Blob programs. They correspond to the basic operations that PEs can perform, such as broadcasts and transfers, as well as some interpreter specific operations, such as *Show* which marks a value to be displayed in the GUI, but otherwise has no effect on the state of the medium. Procedures are used to create more complex instructions by combining basic instructions and other procedures. They are implemented as a list of instructions, and the `exec()` method simply executes the instructions in order, moving on to the next instruction only when the current one has finished executing. Ultimately, the interpreter simply executes a given instruction on loop ad-vitam eternam, making the initial state of the medium (i.e. the initial values of the variables in the program) evolve according to the program.

It is important to note, however, that not all variables in the program are useful to describe the state of the program at the start of an iteration of the loop. Many variables are used temporarily to store intermediate results. The interpreter makes it possible to mark those variables as temporary. The references to the fields contained in a temporary variable are set to *null* at the end of the procedure that created them, allowing Java's garbage collector to free the memory used by those fields. While this may not be important for small programs, given that the number of PEs in a medium can be very large, it is crucial to be able to free memory as soon as possible to avoid running out of memory.

2.2.2 Blob language

Since the Blob language is relatively simple, let's use a small example of a Blob program to illustrate its syntax and semantics. The following program implements a simple algorithm that allows a single starting particle to spread in the medium. Note that the program is not meant to be efficient, but rather to illustrate the features of the Blob language. In the same light, the comments are meant to explain said features, and not to explain the algorithm itself.

```
// A standard Java class
public class Grow {
```

A *BoolVRef* is a wrapper for a boolean field over the vertices. Since the fields are meant to evolve across iteration, this allows us to keep a constant reference to pass around the program

while new fields replace the stale ones. Here, we store the current growth of the particle.

```
BoolVRef grow;
```

The constructor is used to define the initial state of the program. Here, we call various methods which are not part of the Blob instruction set, such as “Math.random()”. This is not an issue as the constructor is only executed once before the interpretation of the program, and does not interfere with the execution past the initial configuration.

```
public Grow(Medium medium) {
    int randIndex = (int) (Math.random() * medium.vertices.size());
    Vertex seed = (Vertex) medium.vertices.toArray()[randIndex];

    // Create a boolean over the vertices that is false everywhere
    BoolV seedField = BoolV.zeros();

    // Set a single vertex in the field (the seed) to true
    BoolV.setBit(seedField, seed, true);

    // Assigns the field to grow as the initial state of the program
    grow = new BoolVRef(seedField);
}
```

A Blob method is defined as an inner class extending the Procedure abstract class. The method body is defined in the constructor of the class. This method enables vertex-to-vertex communication by swapping to two *Ves* pointing to each edge.

Similarly to the outer class, the constructor of the Send class is executed once per call to the Blob method, before the actual execution of the algorithm. This enables the use of non-Blob methods to set up the initial state, which can be useful to define directly here values that should be reused throughout the execution.

```
private class Send extends Procedure {
    public Send(BoolVRef ve) {
        // Boolean fields used to store intermediate results.
        // The use of tmp(...) makes the fields contained in the refs open
        // to garbage collection once we finish executing this method.
        BoolEvRef ev = tmp(new BoolEvRef());
        BoolEfRef ef = tmp(new BoolEfRef());

        // Blob instructions
        // These methods add the corresponding instructions to the execution
        // list of this procedure.
        transfer(ve, ev);
        rotCW(ev, ef);
        rotCW(ef, ev);
        transfer(ev, ve);
    }
}
```

```

    }
}

// The method that performs the spread.
private class Spread extends Procedure {
    public Spread() {
        BoolVRef start = tmp(new BoolVRef());
        BoolVeRef ve = tmp(new BoolVeRef());

        set(grow, start);
        broadcast(grow, ve);

        // Since the send method is user defined and not part of the built-in
        // instruction set, it needs to be passed through a call(...).
        call(new Send(ve));

        redOr(ve, grow);
        or(start, grow, grow);
    }
}
}

```

Another interesting feature of the Blob language is the slight metaprogramming capabilities it provides. Let's consider the following example, which implements a simple addition of two integers.

An IntV is an object that stores an integer as a list of boolean fields over the vertices of the medium. An IntVRef is a wrapper for an IntV, with a similar purpose as the BoolVRef in the previous example. The following class implements a simple addition of two IntVs, and stores the result in a third IntV.

```

class AddV extends Procedure {
    public AddV(IntVRef a, IntVRef b, IntVRef res) {

```

Notice the use of an integer class member "n", despite the fact that the CA cannot directly manipulate integers. This integer is only accessed during the construction of the instruction, never by the interpreter itself. Here, we use it to check that all three IntVs have the same size, and throw an exception at construction time if they don't.

```

        if (a.get().n != b.get().n || a.get().n != res.get().n)
            throw new IllegalArgumentException("Cannot add IntVs of different sizes.");

        IntVRef carry = tmp(new IntVRef(new IntV(a.get().n)));
        IntVRef tmp = tmp(new IntVRef(new IntV(a.get().n)));
        set(a, res);
        set(b, tmp);

```

The addition is performed bit by bit, starting from the least significant bit. We use a for loop to iterate over the bits of the IntVs, despite the fact that the CA cannot directly perform for loops. This is an example of the metaprogramming capabilities of the Blob language, as the for loop is only executed once during the construction of the instruction, and not by the interpreter itself. This means that the interpreter will only see a sequence of instructions that perform the addition, and not the for loop itself. That sequence of instructions is automatically generated at construction time to suit the size of the IntVs. This means that this single AddV class can be used to add IntVs of any size, without having to write a separate class for each size.

```

    for (int i = 0; i <= a.get().n; i++){
        and(res, tmp, carry);
        xor(res, tmp, res);
        lShift(carry, tmp, 1);
    }
}
}

```

For a complete list of the built-in Blob instructions, we refer the reader to [Appendix A](#).

2.2.3 Bit packing and Parallelization

The field programming paradigm forms the basis of the parallelization of basic instructions, which is a key feature of the interpreter. Since the number of PEs in a medium can be very large, it is important to be able to execute instructions in parallel as much as possible. This is achieved by expressing a field of boolean values as an array of integers, where each PE is assigned a single bit (instead of a full byte) representing its value (*note : it is possible for a bit to be unused, but all PEs must have exactly one corresponding bit, and no bit can be attributed to more than one PE*). With a clever organization of this array, it is possible to greatly reduce the number of operations needed to execute an instruction on all PEs of a given type, by performing bitwise operations on the integers in the array.

Regrettably, the parallelization algorithm fell in a strange spot between last year's work and this year's work. The algorithm was fully designed last year, but was not implemented because of time constraints. This year, I implemented the algorithm, but it is no longer the main focus of the internship, and thus I never took the time to properly benchmark it. As a result, I cannot provide any meaningful results regarding the performance of the parallelization algorithm. I am currently considering to benchmark it on my personal time once the internship is over, but I cannot make any promises regarding this. Fortunately, the interpreter is built so that swapping the internal representation of the fields is easy, and thus it should not prove too difficult to implement an un-parallelized version of the interpreter to compare against.

2.3 Graphical User Interface

To facilitate the visualization of the execution of Blob programs, I developed a simple graphical user interface (GUI) using JavaFX. The GUI allows users to load a medium and a Blob

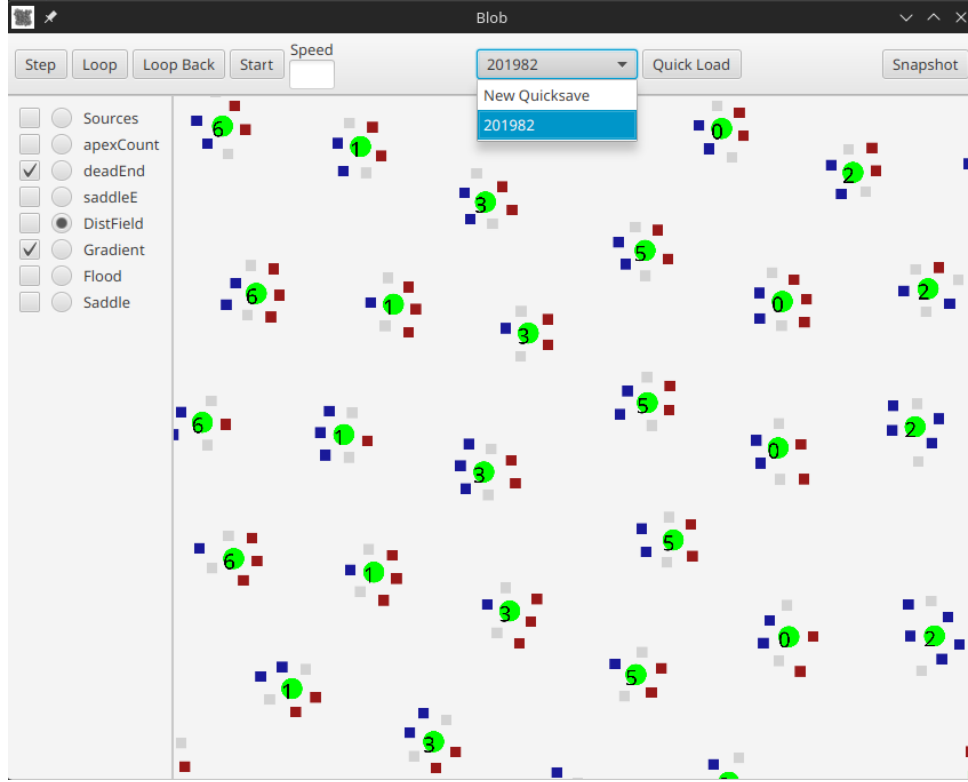


Figure 2.1: A screenshot of the GUI.

program, and to visualize and interact with the evolution of the medium as the program is executed.

It allows users to visualize the state of the medium at any point in time, and to choose which fields to display. Values of fields can be inspected either through a color map, or by displaying the text value of each PE in the medium. The GUI also allows users to take snapshots of the medium at any point in time, and to save them as images.

It also provides the user with various tools to control the execution of the program, such as pausing and resuming, and stepping through the execution (either instruction by instruction, iteration by iteration, or at a controllable speed). It is possible to iterate backwards, or to save the state of the medium at any point in time and to load it later to resume the execution from that point. This is particularly useful for debugging Blob programs, as it allows users to easily identify and isolate the source of any issues that may arise during the execution.

2.4 Preliminary results

Grow

The grow algorithm is a simple algorithm that allows a single starting particle to spread in the medium. It is the first algorithm I implemented in the new framework, and served as a test case for the interpreter and the GUI. It is also a simple example of how to use the Blob language to implement a basic algorithm.

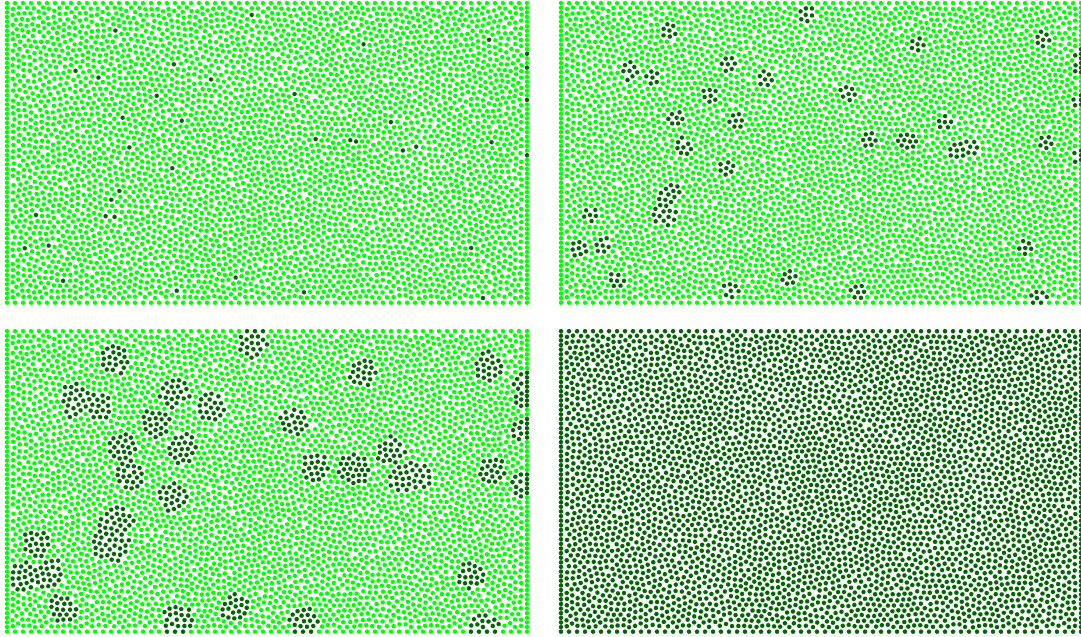


Figure 2.2: A sequence of screenshots of the grow algorithm.

Static Voronoï

The static Voronoï algorithm (as described in 1.3.2) is a simple algorithm that computes the Voronoï diagram of a set of static particles in the medium. It served as a test case for the mirroring of the medium along its borders, as the blobs would stop growing once they detected their own reflection.

Random Field

While the Blob paradigm is deterministic by nature, it is possible to introduce randomness in the initial state of the medium. The random field aims to propagate this randomness through time by repeatedly XORing the values of each PE with the values of its neighbors. This algorithm is useful for generating pseudo-random fields of values in the medium, which can be used for various purposes, such as random walks.

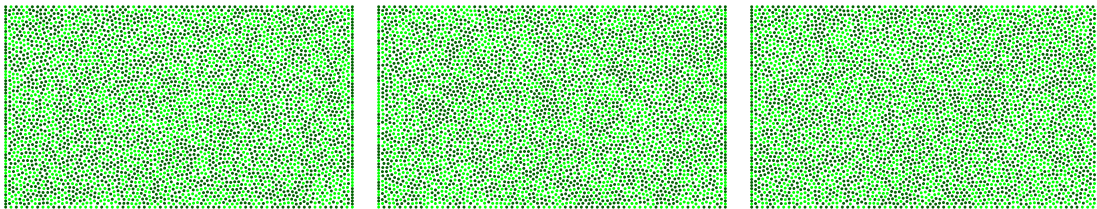


Figure 2.3: A sequence of screenshots of the random field algorithm.

Flies

A fly is a quasi-particle (i.e. a V-blob of size 1, 2 or 3) that randomly moves in the medium. It is the starting point for the homogenization algorithm, as we want to use quasi-particles to represent the particles that we want to spread evenly in the medium. In the hexagonal case, the

use of quasi-particles is necessary to converge, as otherwise the particle could get stuck going back and forth between two positions or three positions if its optimal position happened to fall on an edge or a face.

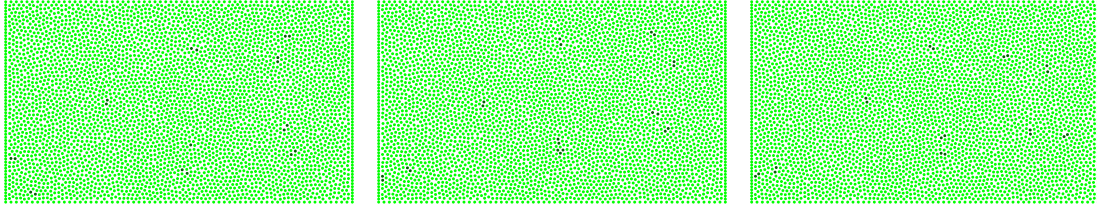


Figure 2.4: A sequence of screenshots of the flies algorithm.

2.5 Toward a solution to the homogenization problem

Given a set of particles in a homogeneous isotropic medium, the homogenization problem consists in spreading the particles evenly throughout the medium. Particles are said to be evenly spread if the minimal distance between any two particles is maximized. This is the same definition I used to homogenize the media themselves, following the approach of [6]. Additionally, we want a finite-state solution, which adds a lot of complexity to the problem.

To solve this problem, we start from a simple idea : we increase the minimal by having each particle move away from its closest neighbor. Despite the simplicity of this idea, it is not trivial to implement. We use the following approach, which is the one we use in the hexagonal case, and which we believe can be adapted here.

We start by computing the dynamic distance field of the particles. It is an integer field that assigns to each *Vertex* its distance to the closest particle. Even though the distance can be arbitrarily large, we can use a finite state approximation for our purpose (see 2.5.1). While such a field may appear to trivialize the problem, it is important to notice that the closest particle to a given particle is itself, and not one of its neighbors. This means that to use the distance field directly, we would need one field per particle, which is not feasible.

Instead, we build the Voronoï diagram of the particles as an intermediate step. Because an edge of the Voronoï diagram is close to a particle if the particle on the other side is also close, we can simply have each particle move away from the borders of its Voronoï cell, or equivalently, toward its barycenter. We only need two distance fields : one computing the distance from the particles to form the Voronoï diagram, and one computing the distance from Voronoï edges and moving the particles.

However, because information travel time scales linearly with distance, it is difficult to maintain a valid Voronoï diagram as the particles move. To solve this issue, we compute the Gabriel centers of the particles as an intermediate to anchor the Voronoï diagram from. A Gabriel center is a special point that is part of the Voronoï diagram (see 2.5.2), but is also much easier to compute in the hexagonal case. We believe that their use will also simplify the homogeneous isotropic case, but this is still a work in progress.

To summarize, the steps of our approach are as follows :

1. Compute the dynamic distance field of the particles

2. Compute the Gabriel centers of the particles based on the distance field
3. Construct the Voronoï diagram of the particles based on the Gabriel centers
4. Compute the distance field of the Voronoï
5. Move the particles to the barycenter of their Voronoï cell

2.5.1 Dynamic distance

We first compute the dynamic distance field of the particles, which is a combination of two fields :

1. An IntV that stores the distance to the closest particle.
2. An IntVe that stores the gradient of the distance field, i.e. how much the distance will change if we move to a neighboring vertex.

We call the distance field "dynamic" because it is updated at each iteration of the algorithm, as the particles move. This allows the particles to always have an up-to-date view of their surroundings, and to make informed decisions about where to move next.

Interestingly, at least in the hexagonal case, and probably in our case as well, we do not need to compute distance itself to compute the Gabriel centers, but only the gradient of the distance field. This is because the Gabriel center of a particle is defined as the point that is equidistant from its closest neighbors, and thus can be computed directly from the gradient of the distance field.

This means that we can store the distance modulo some constant (usually 7 here), which allows us to compute the full gradient using only a finite amount of states. This is a straightforward application of [4].

2.5.2 Gabriel centers

For the purpose of this section, we give a brief overview of the concept of Gabriel centers. For a more detailed explanation, we refer the reader to [3].

In the euclidian plane, the Gabriel graph of a set of points is a subgraph of its Delaunay triangulation. An edge between two points is in the Gabriel graph if and only if 1. it is an edge in the Delaunay triangulation, and 2. it does not intersect with any Voronoï cell of any other point. The point at the center of the edge is called a "Gabriel center" of the set of points. Because of the duality relationship between the Delaunay triangulation and the Voronoï diagram, the Gabriel centers are also the centers of some edges of the Voronoï diagram. The Gabriel is interesting because it formally captures the notion of "closest neighbors" in a set of points, which is essential for our approach to the homogenization problem.

In the case of Blob computing, we do not work in the euclidian plane, but rather in a planar graph. Because of this, we use the generalized definition presented in [3], which defines the Gabriel graph of a set of points in an arbitrary metric space. The main differences with the euclidian case are the following :

- The Gabriel graph may contain multiple paths between two points (where a path is akin to an edge in the euclidian case), as there can be multiple shortest paths between two points in a graph. In this case, the Gabriel centers are the points at the center of each of these paths.
- A Gabriel center can be either a vertex or an edge of the medium, depending on whether the path between two points has an odd or even length.

Formally, c is a Gabriel center of a set of generators S if for some $p, q \in S$, for $r = \frac{1}{2}d(p, q)$, there exists a ball $B = \mathcal{B}(c, r)$ such that there is a partition (S_0, S_1) of $S \cap B$ s.t. $d(S_0, S_1) = 2r$.

Intuitively, we separate the generators into two groups of generators that are both sufficiently far from each other to be distinct, and sufficiently close to be considered "close neighbors" relative to the other generators. c is then the point at the center of the two groups.

2.5.3 Computation in the hexagonal lattice case, and challenges in the homogeneous isotropic case

An important property of the Gabriel centers in the hexagonal case is that their position can be computed independently of the Gabriel graph itself. In this case, the computation is fairly straightforward. Given a point c , we want to determine whether it is a Gabriel center or not. Let's notice the following :

- The gradient field can only decrease in two distinct directions around c if there are at least two particles at some distance r from c ,
- There cannot be any particles at distance less than r from c , otherwise the gradient would point toward them instead.
- If the distance between the particles is too small, then the gradient field will not be able to distinguish between them.

This means that c is a Gabriel center if and only if there are at least two non-adjacent V s around c that are decreasing in the gradient field. This is a local property that can be easily computed by the PEs around c , and thus can be implemented as a Blob program.

This approach faces a significant difficulty in the homogeneous isotropic case. A big advantage of the regular lattice is that the distance field propagates "cleanly" in the medium, while our topology is just irregular enough for it to cause issues. Most notably, in some configurations it is possible for the gradient to "split" despite only pointing toward a single particle.

This split is due to localized variations in the "propagation speed" of the distance field. 2.5a shows a case where the gradient splits at a vertex. Around the central vertex, we observe that the gradient pointing toward the nearest particle (marked with blue squares) is split in the middle by a null gradient (marked with grey squares) despite the fact that there is a single particle in the medium. This happens when two vertices are placed consecutively in the direction of the gradient while still being at the same distance from the particle, which is a result of the distance updating slightly slower on the side of those two vertices. 2.5b illustrates the same phenomenon, except that the gradient splits at an edge.

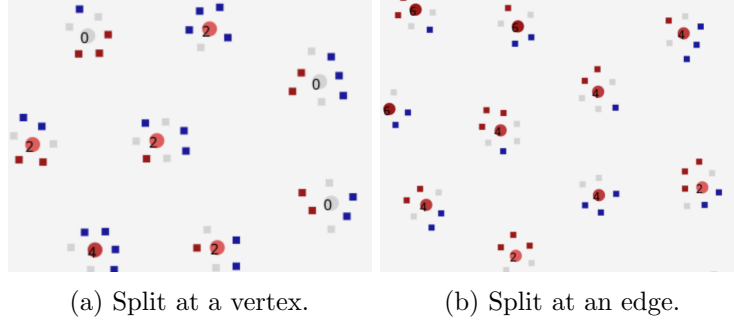


Figure 2.5: Gradient splitting in a homogeneous isotropic medium. A positive gradient is red, a neutral gradient is grey, and a negative gradient is blue.

2.5.4 Current attempts at solving the problem

Correction of the hexagonal approach

My first approach was to try and detect the configurations that cause the gradient to split, and to correct them. While it is fairly easy to detect the configurations that cause the gradient to split, it is difficult to distinguish between a them and configurations that naturally arise from the presence of adjacent Gabriel centers.

As of now, it is not clear whether this approach fails because of the nature of the media, or because said media are not homogeneous enough. While producing a more homogeneous medium is possible, it would be a time-consuming process, and it would further limit the generality of the model. Thus, I decided to explore other approaches.

Gradient smoothing

My second approach was to try and smooth the gradient field by "locally averaging" its direction. Ultimately, I decided this was not worth pursuing. Due to the isotropic nature of the medium, it is difficult to work on direction-related properties, and after a few days of work I had not made any progress.

Flooding algorithm

My third and current approach is to use a "flooding" algorithm to compute the Gabriel centers. The idea is to repeatedly flood the medium in the direction of the gradient field on a limited distance, and to mark the points where the floods meet as Gabriel centers. This approach is more robust to the irregularities of the medium, but is also more complex to implement, and requires a more sophisticated Blob program.

The flooding algorithm is as follows :

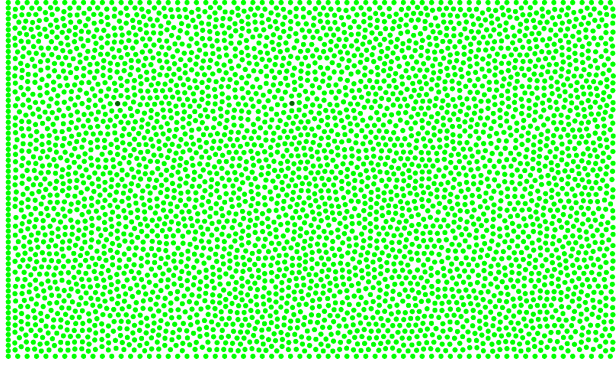
1. Choose a flooding period T .
2. Select every vertex at distance $n \cdot T, n \in \mathbb{N}$. These vertices will be the starting points of the floods, and form concentric near-circle around each particle.
3. For each starting vertex, also select its neighbors if the gradient towards them is positive or neutral. Repeat this process $T - 4$ more times (until bands of width $T - 2$ are selected,

leaving unselected bands of width 1 in between). This will create a "flooding" of the medium in the direction of the gradient field.

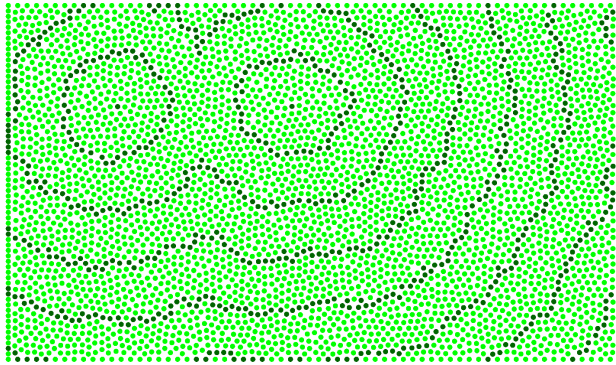
4. For each vertex, if the flooded area around it has a radius of at least $T - 3$, then consider it a Gabriel center.
5. Repeat steps 2-4 with starting points at distance $n \cdot T + 1, n \in \mathbb{N}$, then $n \cdot T + 2$, and so on until $n \cdot T + T - 1$.

2.6 shows an example of the flooding algorithm in action.

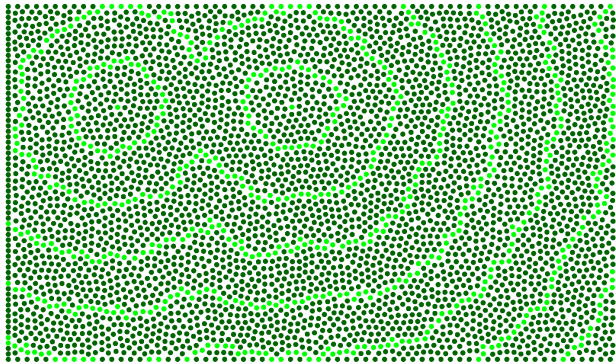
While this approach seems promising, it is still in its early stages. As of now, I detect either too many or too few Gabriel centers, and I am working on refining the algorithm to improve its accuracy. Regarding the case where too many centers are detected, I am currently deciding whether to eliminate the "false" centers at all, as they seem to align somewhat with the Voronoï diagram which we are trying to build anyway. Finally, because the flooding relies on "flooding width" for the detection, it fails to find Gabriel centers that are closer than T to a particle.



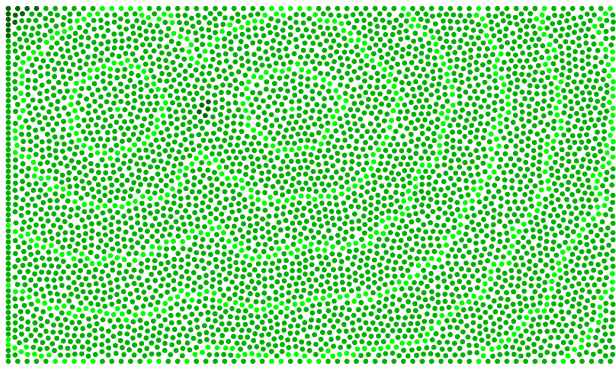
(a) The set of particles.



(b) The starting points of the floods.



(c) The flooding of the medium.



(d) The detected Gabriel centers, where the flood is thick enough.

Figure 2.6: One iteration of the flooding algorithm on 2 particles with $T = 7$.

Chapter 3

Evaluation and discussion

3.1 Evaluation of the performed work

The work performed during this internship has been successful in many ways. The new Blob computing framework that I developed is functional and allows for the execution of Blob programs on homogeneous isotropic media. The graphical user interface is also functional and provides a convenient way to visualize and interact with the execution of Blob programs.

The homogenization work is still in progress and more exploratory in nature, but it has helped develop a useful intuition about the topic and its challenges, and has yielded some promising leads toward a solution. While the problem is not yet solved, this has laid a solid foundation for the continuation of this internship.

Still, the detection of Gabriel center is a crucial step in the homogenization algorithm, and although I do feel that success is achievable, it is not yet clear whether the current approaches of correction and flooding will bear fruit.

3.2 Impact of the internship

The Blob computing framework that I developed during this internship, along with the GUI to exploit it, both represent a worthwhile contribution in my eyes, if only for their simplicity and practicality. It is my hope that they would prove useful for future research in the field of Blob computing, and that it will enable the exploration of new applications and algorithms in this area.

Additionally, while this internship did not result in a complete solution to the homogenization problem, it did provide insight into various approaches to a solution, and highlighted the challenges that arise when working with homogeneous isotropic media. Furthermore, as a bit more than a quarter of the internship is still to be completed, I am confident that we will be able to make significant progress on this problem in the remaining time.

3.3 Reflections on the internship

3.3.1 Difficulties encountered and reactions

The main difficulty I encountered during this internship was the jump in complexity from the hexagonal lattice case to the homogeneous isotropic case. While the two cases are similar in many ways, the irregularities of the homogeneous isotropic medium caused many issues that were not present in the hexagonal case. The isotropy is especially challenging to work with, as it destroys many of the properties that we relied on in the hexagonal case such as the ability to perform symmetries around vertices, renders the medium highly irregular, leading to the splitting of the gradient field, and makes it difficult to work with direction-related properties.

Of course, this difficulty was not unexpected, and is the reason why this internship was so attractive to me in the first place. Still, it led to a complete rethinking of the approach used in the hexagonal case, and forced me to explore new ideas and techniques. While this was frustrating at times, it was also a valuable learning experience that helped me develop my problem-solving skills and my ability to think creatively.

3.3.2 Skills acquired and lessons learned

This internship helped me develop many skills that will be useful in my future career.

On the technical side, I feel that I have greatly improved my programming skills, particularly in Java. Building the interpreter and GUI from scratch pushed me to hone my skills in software design and architecture, and I now have a much deeper understanding of Java's type system and overall capabilities.

On the scientific side, I of course developed a much sharper understanding of Blob computing, but more generally, I improved at developing, expanding, critiquing and defending scientific ideas, both my own and those of others. I also improved my ability to find, read and understand scientific papers, and to extract the relevant information from them.

3.3.3 Career plans after the internship

After this internship, I plan on working for a few months in education, mostly personal tutoring, before pursuing a PhD in distributed computing. I am particularly interested in amorphous networks, such as the ones studied in this internship, as well as networks of mobile agents, such as those studied in swarm robotics.

Chapter 4

Conclusion

Over the course of this internship, I have developed a new Blob computing framework that allows for the execution of Blob programs on homogeneous isotropic media. I have also developed a graphical user interface that allows users to visualize and interact with the execution of Blob programs. This work established a practical foundation for experimenting with Blob computations in a more general setting than the regular hexagonal lattice used in previous work.

The internship also explored the homogenization problem. While a complete solution to the problem was not achieved, the work performed during this internship has provided insight into various approaches to a solution, and highlighted the stark challenges posed by the isotropic nature of the medium. The work performed during this internship has laid a solid foundation for the continuation of this research, and I am confident that significant progress can be made in the remaining time.

Overall, this internship has been a valuable learning experience that has helped me develop my technical and scientific skills, and has provided me with a deeper understanding of Blob computing and the challenges of working with homogeneous isotropic media. It has also led me to develop a greater appreciation for the importance of creativity and perseverance in scientific research, and has reinforced my desire to pursue a career in this field.

4.1 Acknowledgements

I would like to thank my supervisor, Frédéric Gruau, for his guidance and support throughout the internship. His expertise in the field of Blob computing was invaluable in helping me navigate the challenges of the project. His encouragement and critique helped nurture my scientific curiosity, and were instrumental in helping me develop my skills as a researcher.

References

- [1] Frederic Gruau. “Modular programming of computing media using spatial types, for artificial physics”. In: (2019). arXiv: [1904.05787](https://arxiv.org/abs/1904.05787) [cs.DC]. URL: <https://arxiv.org/abs/1904.05787>.
- [2] Frédéric Gruau, Christine Eisenbeis, and Luidnel Maignan. “The foundation of self-developing blob machines for spatial computing”. In: *Physica D: Nonlinear Phenomena* 237.9 (2008). Novel Computing Paradigms: Quo Vadis?, pp. 1282–1301. ISSN: 0167-2789. DOI: <https://doi.org/10.1016/j.physd.2008.03.046>. URL: <https://www.sciencedirect.com/science/article/pii/S0167278908001255>.
- [3] Luidnel Maignan and Frédéric Gruau. “Gabriel Graphs in Arbitrary Metric Space and their Cellular Automaton for Many Grids”. In: *ACM Trans. Auton. Adapt. Syst.* 6.2 (June 2011). ISSN: 1556-4665. DOI: [10.1145/1968513.1968515](https://doi.org/10.1145/1968513.1968515). URL: <https://doi.org/10.1145/1968513.1968515>.
- [4] Luidnel Maignan and Frédéric Gruau. “Integer Gradient for Cellular Automata: Principle and Examples”. In: Nov. 2008, pp. 321–325. DOI: [10.1109/SASOW.2008.52](https://doi.org/10.1109/SASOW.2008.52).
- [5] S. Reboh et al. “NanoStack Transistor Architecture for CMOS 7A Node and Beyond”. In: *2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 2025, pp. 1–3. DOI: [10.23919/VLSITechnologyandCir65189.2025.11074866](https://doi.org/10.23919/VLSITechnologyandCir65189.2025.11074866).
- [6] Thomas Schlömer, Daniel Heck, and Oliver Deussen. “Farthest-point optimized point sets with maximized minimum distance”. In: *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics*. HPG ’11. Vancouver, British Columbia, Canada: Association for Computing Machinery, 2011, pp. 135–142. ISBN: 9781450308960. DOI: [10.1145/2018323.2018345](https://doi.org/10.1145/2018323.2018345). URL: <https://doi.org/10.1145/2018323.2018345>.

Appendices

A List of built-in Blob instructions

Special instructions

Instructions that do not affect the execution but can be used by the interpreter.

- **print(String s):** prints s to the console whenever it is reached.
- **show(Ref r):** indicates that the value of r should be displayed by the UI.
- **snapshot():** signals to the UI that the current display should be saved as an image.

Boolean instructions

Instructions that only affect boolean fields.

- **And(BoolField a , BoolField b , BoolField res):** stores a AND b in res .
- **Broadcast(BoolFieldS a , BoolFieldT res):** Broadcasts a to res .
- **If(BoolField $cond$, BoolField t , BoolField f , BoolField res):** Stores the value of t (resp. f) in res where $cond$ is true (resp. false).
- **Not(BoolField a , BoolField res):** stores NOT a in res .
- **Or(BoolField a , BoolField b , BoolField res):** stores a OR b in res .
- **RedAnd(BoolFieldT a , BoolFieldS res):** stores the AND reduction of a in res .
- **RedOr(BoolFieldT a , BoolFieldS res):** stores the OR reduction of a in res .
- **RedStack(BoolFieldT a , BoolFieldS[] res):** converts a transfer field a into an array of simplicial fields res .
IMPORTANT: while this is not a legal Blob instruction, it greatly simplifies the implementation of some other legal Blob instructions. Thus, I implemented it to save on development time, but it should be removed in the future.
- **RedXor(BoolFieldT a , BoolFieldS res):** stores the XOR reduction of a in res .
- **RotCCW(BoolFieldT a , BoolFieldT res):** rotates a counter-clockwise into res .
- **RotCW(BoolFieldT a , BoolFieldT res):** rotates a clockwise into res .

- **Set**(BoolField a , BoolField res): copies a to res .
- **Transfer**(BoolFieldT a , BoolFieldT res): transfers a to its dual field res .
- **Xor**(BoolField a , BoolField b , BoolField res): stores a XOR b in res .

Integer instructions

Instructions that affect integer fields, possibly along with boolean fields.

- **Add**(IntField a , IntField b , IntField res): stores $a + b$ in res .
- **And**(IntField a , IntField b , IntField res): stores a AND b (bitwise) in res .
- **BoolToInt**(BoolField a , IntField res): sets res to 1 where a is true, and 0 where a is false.
- **Broadcast**(IntFieldS a , IntFieldT res): Broadcasts a to res .
- **Eq**(IntField a , IntField b , BoolField res): stores $a == b$ in res .
- **GT**(IntField a , IntField b , BoolField res): stores $a > b$ in res .
- **If**(BoolField $cond$, IntField t , IntField f , IntField res): Stores the value of t (resp. f) in res where $cond$ is true (resp. false).
- **Neg**(IntField a , IntField res): stores $-a$ in res .
- **Not**(IntField a , IntField res): stores NOT a (bitwise) in res .
- **Or**(IntField a , IntField b , IntField res): stores a OR b (bitwise) in res .
- **RedAdd**(IntFieldT a , IntFieldS res): stores the sum reduction of a in res .
- **RedMax**(IntFieldT a , IntFieldS res): stores the max reduction of a in res .
- **RedMin**(IntFieldT a , IntFieldS res): stores the min reduction of a in res .
- **RedStack**(IntFieldT a , IntFieldS[] res): converts a transfer field a into an array of simplicial fields res .
IMPORTANT: see warning in the Boolean version of this instruction.
- **RotCCW**(IntFieldT a , IntFieldT res): rotates a counter-clockwise into res .
- **RotCW**(IntFieldT a , IntFieldT res): rotates a clockwise into res .
- **ScanLeft**(IntField a , BoolField res , (BoolField val , BoolField acc) $\rightarrow$ **Instruction operator**): stores the left scan of the bits of a in res , using the provided binary *operator* to combine values.
- **ScanRight**(IntField a , BoolField res , (BoolField val , BoolField acc) $\rightarrow$ **Instruction operator**): stores the right scan of the bits of a in res , using the provided binary *operator* to combine values.

- **Set(IntField a , IntField res):** copies a to res .
- **ShiftLeft(IntField a , IntField res , int k):** stores $a \ll k$ in res .
- **ShiftRight(IntField a , IntField res , int k):** stores $a \gg k$ in res .
- **Sub(IntField a , IntField b , IntField res):** stores $a - b$ in res .
- **Transfer(IntFieldT a , IntFieldT res):** transfers a to its dual field res .
- **Xor(IntField a , IntField b , IntField res):** stores $a \text{ XOR } b$ (bitwise) in res .